

KALKULATOR PŁACOWY

Aplikacja Webowa

Specyfikacja Wymagań Systemu + Projekt Techniczny

Piotr Szabelski · Martyna Fita · Hubert Witkowski · Borys Frączek · Zbigniew Płatkowski

Spring Boot 3 · Java 21 · PostgreSQL 16 · HTML / CSS / JS · REST API

Kwiecień 2026

Spis Treści

Spis Treści	2
S1. Opis Ogólny Systemu	4
S2. Założenia Systemowe	4
S3. Słownik Pojęć.....	5
S4. Wymagania Funkcjonalne	6
S5. Wymagania Pozafunkcjonalne	8
S6. Diagram Przypadków Użycia	9
1. Cel Dokumentu	10
2. Stos Technologiczny	10
2.1 Kluczowe zależności Maven (pom.xml)	11
3. Architektura Systemu	12
3.1 Bezpieczeństwo	12
4. Model Danych.....	13
4.1 Tabela: users	13
4.2 Tabela: password_reset_tokens (WF.17).....	13
4.3 Tabela: companies.....	13
4.4 Tabela: employees.....	14
4.5 Tabela: absences (WF.25).....	14
4.6 Tabela: payslips	15
4.7 Tabela: gus_life_expectancy	16
4.8 Diagram relacji (ERD)	16
5. Specyfikacja REST API	17
5.1 Autentykacja i reset hasła (WF.17).....	17
5.2 Pracownicy (WF.22, WF.23)	17
5.3 Nieobecności (WF.25)	18
5.4 Paski płacowe (WF.18, WF.19, WF.21, WF.24)	18
5.5 Emerytura (WF.11, WF.12, WF.14, WF.20).....	19
6. Logika Obliczeń Płacowych.....	20
6.1 Składki ZUS – pracownik (stawki 2025).....	20
6.2 Składki ZUS – pracodawca (do kosztu WF.08)	20
6.3 Algorytm obliczenia wynagrodzenia netto.....	20
6.4 Obliczenie emerytury i wykresu (WF.11–14, WF.20).....	21
7. Frontend – HTML / CSS / JavaScript	22
7.1 Ekrany aplikacji	22
7.2 Wykres skumulowanych składek (WF.20 – Chart.js).....	23
8. Proponowana Struktura Projektu.....	24
9. Środowisko i Uruchomienie	26
9.1 application.properties	26

9.2 Uruchomienie	26
10. Termin Realizacji	27
11. Lista Kontrolna przed Oddaniem	27
Pokrycie wymagań	27
Bezpieczeństwo	29
Jakość kodu	29

CZĘŚĆ I – SPECYFIKACJA WYMAGAŃ SYSTEMU

S1. Opis Ogólny Systemu

Przedmiotem niniejszego dokumentu jest specyfikacja wymagań dla aplikacji webowej pełniącej rolę kalkulatora płacowego. System udostępnia pracownikom szczegółowe informacje o ich wynagrodzeniu netto, historii wypłat oraz prognozowanej emeryturze. Pracodawca posiada rozszerzony dostęp umożliwiający wprowadzanie i modyfikację danych niezbędnych do obliczeń.

Dane dotyczące wynagrodzeń stanowią dane wrażliwe w rozumieniu przepisów o ochronie danych osobowych, w związku z czym system musi zapewniać wysoki poziom poufności, integralności i dostępności przetwarzanych informacji.

S2. Założenia Systemowe

- Dane pracownika potrzebne do kalkulacji muszą być przekazane do systemu oraz modyfikowane w razie potrzeby przez pracodawcę.
- System nie obsługuje walut innych niż PLN.
- Obliczenia podatkowe i składowe są zgodne z polskim prawem obowiązującym w momencie wdrożenia i wymagają aktualizacji przy zmianie przepisów.
- Wskaźnik GUS dotyczący dalszego trwania życia jest aktualizowany corocznie na podstawie oficjalnych tabel GUS.
- System nie zastępuje oficjalnych dokumentów kadrowo-płacowych – pełni wyłącznie funkcję informacyjną.

S3. Słownik Pojęć

Pojęcie	Definicja
ZUS	Zakład Ubezpieczeń Społecznych – instytucja zarządzająca systemem ubezpieczeń społecznych w Polsce.
NIP	Numer Identyfikacji Podatkowej – unikalny identyfikator podatkowy nadawany przez urząd skarbowy.
REGON	Krajowy Rejestr Urzędowy Podmiotów Gospodarki Narodowej – numer statystyczny nadawany podmiotom prowadzącym działalność gospodarczą.
Netto	Kwota wynagrodzenia po odliczeniu wszystkich składek ubezpieczeniowych i zaliczki na podatek dochodowy.
Brutto	Kwota wynagrodzenia przed odliczeniem składek i podatku, stanowiąca podstawę kalkulacji.
RODO	Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 w sprawie ochrony osób fizycznych w związku z przetwarzaniem danych osobowych.
RBAC	Role-Based Access Control – mechanizm kontroli dostępu oparty na przypisanych rolach użytkowników.
GUS	Główny Urząd Statystyczny – instytucja publikująca oficjalne tablice dalszego trwania życia, używane do szacowania emerytury.
OFE	Otwarty Fundusz Emerytalny – fundusz gromadzący część składki emerytalnej w ramach II filaru systemu emerytalnego.

S4. Wymagania Funkcjonalne

Sygnatura	Opis Wymagania
WO	Wymagania Ogólne
WO.01	System działa jako samodzielna aplikacja webowa komunikująca się z relacyjną bazą danych pracowników.
WO.02	System udostępnia dwa rodzaje kont: pracodawcy oraz pracownika. W zależności od roli użytkownik posiada dostęp wyłącznie do danych mu przypisanych.
WO.03	Pracownik ma wgląd tylko do własnych danych bez możliwości ich edycji. Pracodawca może przeglądać dane wszystkich pracowników oraz modyfikować dane niezbędne do obliczeń płacowych.
WO.04	System obsługuje wyłącznie walutę PLN.
WO.05	Dane pracownika wymagane do kalkulacji muszą być wprowadzone i aktualizowane przez pracodawcę.
WF	Wymagania Funkcjonalne – Dane Firmy i Pracownika
WF.01	System przechowuje i wyświetla dane firmy (nazwa, NIP, REGON) na pasku płacowym pracownika.
WF.02	System przechowuje dane osobowe pracownika (imię, nazwisko, PESEL, adres) oraz informacje zgłoszone do ZUS (tytuł ubezpieczenia, kody).
WF.03	System umożliwia określenie daty sporządzenia dokumentu oraz okresu, za który wypłacane jest wynagrodzenie.
WF	Wymagania Funkcjonalne – Kalkulator Wynagrodzenia
WF.04	System oblicza składniki wynagrodzenia brutto (wynagrodzenie zasadnicze, premie, dodatki) oraz nalicza odpowiednie składki ZUS po stronie pracownika i pracodawcy.
WF.05	System uwzględnia zasiłki (np. chorobowy, macierzyński) w obliczeniach wynagrodzenia, korygując podstawę składek ZUS.
WF.06	System nalicza zaliczkę na podatek dochodowy zgodnie z obowiązującymi progami podatkowymi i ulgami (np. kwota wolna od podatku, ulga podatkowa).
WF.07	System uwzględnia inne potrącenia pracownicze (np. komornicze, dobrowolne) w obliczeniu kwoty do wypłaty.
WF.08	System oblicza całkowity koszt zatrudnienia po stronie pracodawcy (brutto + składki pracodawcy).
WF.09	System uwzględnia urlopy i nieobecności (płatne i bezpłatne) przy naliczaniu wynagrodzenia za dany okres.
WF.10	System wylicza i wyświetla końcową kwotę wynagrodzenia netto (do wypłaty na rękę).
WF	Wymagania Funkcjonalne – Szacowanie Emerytury
WF.11	System oblicza i wyświetla szacowaną bieżącą wysokość emerytury na podstawie łącznej wartości dotychczas naliczonych i zewidencjonowanych składek emerytalnych pracownika.
WF.12	System oblicza szacowaną przyszłą emeryturę, uwzględniając wiek pracownika, planowany wiek emerytalny oraz prognozowane przyszłe składki przy założeniu stałego wynagrodzenia.

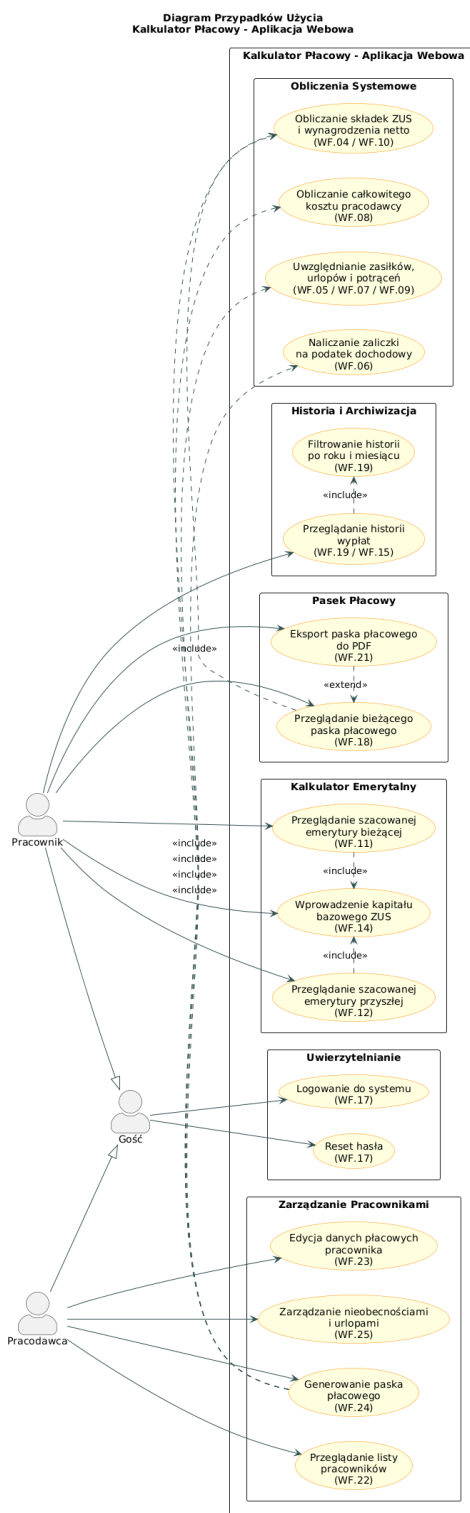
WF.13	System przy wyliczaniu emerytury stosuje wskaźnik GUS dotyczący przeciętnego dalszego trwania życia po osiągnięciu wieku emerytalnego, aktualizowany corocznie.
WF.14	System rozróżnia składki emerytalne gromadzone w ZUS (I filar), wyliczane automatycznie, oraz – opcjonalnie – środki w OFE (II filar), wprowadzane ręcznie przez pracownika. Na podstawie tych danych system umożliwia szacowanie przyszłej wysokości emerytury.
WF Wymagania Funkcjonalne – Historia i Archiwizacja	
WF.15	System przechowuje historię pasków płacowych pracownika i umożliwia przeglądanie danych z poprzednich okresów rozliczeniowych.
WF.16	System umożliwia eksport paska płacowego do formatu PDF.
WF Wymagania Funkcjonalne – Moduł Pracownika	
WF.17	Panel logowania – formularz uwierzytelnienia z obsługą błędów i resetem hasła.
WF.18	Pasek płacowy – widok szczegółowego rozliczenia bieżącego okresu: składniki brutto, potrącenia (ZUS, podatek, inne), kwota netto do wypłaty.
WF.19	Historia wypłat – lista archiwalnych pasków płacowych z możliwością filtrowania po roku i miesiącu.
WF.20	Kalkulator emerytalny – widok z aktualnym szacunkiem emerytury (na dziś i na moment osiągnięcia wieku emerytalnego), wykres skumulowanych składek.
WF.21	Eksport PDF – możliwość pobrania wybranego paska płacowego w formacie PDF.
WF Wymagania Funkcjonalne – Moduł Pracodawcy	
WF.22	Lista pracowników – tabela z możliwością wyszukiwania, sortowania i przejścia do karty pracownika.
WF.23	Karta pracownika – formularz edycji danych płacowych (wynagrodzenie, kody ZUS, ulgi, potrącenia).
WF.24	Generowanie paska płacowego – wybór pracownika i okresu, inicjowanie obliczeń, podgląd wyniku.
WF.25	Zarządzanie nieobecnościami – wprowadzanie urlopów, zwolnień lekarskich i innych nieobecności.

S5. Wymagania Pozafunkcjonalne

Sygnatura	Opis Wymagania
WNF.01	Czas generowania paska płacowego nie może przekraczać 10 sekund od momentu żądania.
WNF.02	Komunikacja między przeglądarką a serwerem musi odbywać się wyłącznie za pośrednictwem szyfrowanego protokołu HTTPS (TLS 1.2 lub wyższy).
WNF.03	Hasła użytkowników muszą być przechowywane w bazie danych w postaci zahashowanej; zabronione jest przechowywanie haseł w postaci jawnej.
WNF.04	System nie przechowuje danych dotyczących środków zgromadzonych w OFE wprowadzonych przez użytkownika. Dane te są wykorzystywane wyłącznie w ramach bieżącej sesji do wykonania symulacji i nie są zapisywane w bazie danych ani w żadnej trwałej formie.
WNF.05	Dostęp do danych wrażliwych (dane osobowe, wynagrodzenia) musi być kontrolowany mechanizmem RBAC; pracownik nie może uzyskać dostępu do danych innego pracownika.
WNF.06	Interfejs użytkownika musi być intuicyjny i czytelny; obsługa podstawowych funkcji nie powinna wymagać szkolenia.
WNF.07	System musi działać poprawnie w najnowszych wersjach przeglądarek: Chrome, Firefox, Edge oraz Safari.
WNF.08	Dane płacowe muszą być archiwizowane i nieusuwalne przez okres co najmniej 5 lat zgodnie z przepisami prawa pracy.
WNF.09	System musi być zgodny z przepisami RODO w zakresie przetwarzania i przechowywania danych osobowych.
WNF.10	Wszelkie obliczenia płacowe muszą być zgodne z aktualnie obowiązującymi przepisami podatkowymi i składkowymi (ustawa o PIT, ustawa o systemie ubezpieczeń społecznych).

S6. Diagram Przypadków Użycia

Poniższy diagram ilustruje przypadki użycia systemu z podziałem na aktora Pracownik, Gość (niezalogowany) i Pracodawca. Odniesienia do sygnatur wymagań podano w nawiasach.



Rysunek 1. Diagram przypadków użycia – Kalkulator Płacowy

CZĘŚĆ II – PROJEKT TECHNICZNY

1. Cel Dokumentu

Niniejsza część stanowi projekt techniczny systemu skierowany do grupy implementacyjnej. Zawiera stos technologiczny, architekturę, model danych, specyfikację REST API, logikę obliczeń, proponowaną strukturę projektu i wymagania środowiskowe. Wszystkie decyzje projektowe powiązane są z sygnaturami wymagań z Części I.

2. Stos Technologiczny

Warstwa	Technologia	Uzasadnienie
Backend	Java 21 + Spring Boot 3	Stabilny LTS, Spring Security wbudowany, duża dokumentacja
API	REST (JSON)	Czytelna separacja frontendu i backendu, testowanie przez Postman
Frontend	HTML5 + CSS3 + Vanilla JavaScript	Brak frameworka JS – mniej zależności, prostszy debugging
Stylowanie	Bootstrap 5	Gotowe komponenty, responsywność bez pisania CSS od zera
Wykresy (WF.20)	Chart.js (CDN)	Lekka biblioteka JS do wykresu skumulowanych składek emerytalnych
ORM / baza danych	Hibernate / JPA + PostgreSQL 16	Standard w Javie enterprise, deklaratywne mapowanie encji
Migracje schematu	Flyway	Wersjonowane pliki SQL uruchamiane automatycznie przy starcie
Bezpieczeństwo	Spring Security (sesja HTTP)	Wbudowany w Spring Boot, łatwa konfiguracja ról RBAC
Hashowanie haseł	BCryptPasswordEncoder	Wbudowany w Spring Security, strength=12
Reset hasła (WF.17)	Token jednorazowy w bazie + e-mail	Tabela password_reset_tokens, Spring Mail (SMTP)
Generowanie PDF (WF.16/WF.21)	iText 7 Community	Biblioteka Java – generowanie PDF po stronie serwera
Build / zależności	Maven 3	Standard w projektach Spring Boot, plik pom.xml
Testy	JUnit 5 + Mockito + MockMvc	Standardowy zestaw dla Spring Boot

2.1 Kluczowe zależności Maven (pom.xml)

```
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId></dependency>
  <dependency><groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId></dependency>
  <dependency><groupId>com.itextpdf</groupId>
    <artifactId>itext7-
core</artifactId><version>7.2.5</version><type>pom</type></dependency>
  <dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-
test</artifactId><scope>test</scope></dependency>
</dependencies>
```

3. Architektura Systemu

Aplikacja oparta na wzorcu MVC w Spring Boot. Frontend (HTML/CSS/JS) komunikuje się z backendem wyłącznie przez REST API. Dane zwracane jako JSON.

Warstwa	Klasy Spring	Odpowiedzialność
Kontrolery	@RestController	Odbieranie żądań HTTP, walidacja wejścia, zwracanie JSON
Serwisy	@Service + @Transactional	Logika biznesowa: obliczenia ZUS, PIT, emerytura, PDF
Repozytoria	JpaRepository<E, ID>	Dostęp do bazy przez Hibernate/JPA, zapytania JPQL
Encje	@Entity + @Table	Mapowanie tabel PostgreSQL na klasy Java
DTO	Klasy POJO / Record	Obiekty żądań i odpowiedzi REST (oddzielone od encji)
Bezpieczeństwo	SecurityConfig	Autoryzacja sesyjna, RBAC przez role użytkownika

3.1 Bezpieczeństwo

Wymaganie	Implementacja	Sygnatura
Hashowanie haseł	BCryptPasswordEncoder strength=12, nigdy jawne hasło w bazie	WNF.03
RBAC – role	@PreAuthorize("hasRole('EMPLOYER')") na serwisie/kontrolerze	WNF.05
Pracownik widzi tylko swoje dane	Weryfikacja: employee.getUserId().equals(currentUserId)	WO.03
Reset hasła	Token UUID w tabeli password_reset_tokens, ważność 1h, Spring Mail	WF.17
HTTPS	server.ssl.* w application.properties lub Nginx reverse proxy	WNF.02
CSRF	Spring Security CSRF token – przy REST API .csrf().disable()	WNF.05
Dane OFE nie zapisywane	Parametry OFE tylko w DTO, nie mapowane do encji	WNF.04
Archiwizacja pasków	Brak DELETE na tabeli payslips – tylko zapis i odczyt	WNF.08

4. Model Danych

Schemat bazy PostgreSQL. Migracje Flyway w src/main/resources/db/migration/.

4.1 Tabela: users

Kolumna	Typ SQL	Ograniczenie	Opis
id	BIGSERIAL	PK	Auto-inkrementowany identyfikator
email	VARCHAR(255)	UNIQUE NOT NULL	Login użytkownika
password_hash	VARCHAR(255)	NOT NULL	BCrypt hash – nigdy jawne hasło (WNF.03)
role	VARCHAR(20)	NOT NULL	'EMPLOYER' lub 'EMPLOYEE'
is_active	BOOLEAN	DEFAULT TRUE	Soft delete – nieaktywne konta zablokowane
created_at	TIMESTAMPTZ	DEFAULT NOW()	Data założenia konta

4.2 Tabela: password_reset_tokens (WF.17)

Kolumna	Typ SQL	Ograniczenie	Opis
id	BIGSERIAL	PK	Identyfikator
user_id	BIGINT	FK → users	Użytkownik, który prosi o reset
token	VARCHAR(36)	UNIQUE NOT NULL	UUID jednorazowy token
expires_at	TIMESTAMPTZ	NOT NULL	Ważność tokenu – 1 godzina od wygenerowania
used	BOOLEAN	DEFAULT FALSE	Czy token już został użyty

4.3 Tabela: companies

Kolumna	Typ SQL	Ograniczenie	Opis
id	BIGSERIAL	PK	Identyfikator firmy
name	VARCHAR(255)	NOT NULL	Nazwa firmy (WF.01)
nip	VARCHAR(10)	UNIQUE	NIP firmy – 10 cyfr
regon	VARCHAR(14)		REGON firmy (WF.01)
owner_id	BIGINT	FK → users	Pracodawca-właściciel firmy

4.4 Tabela: employees

Kolumna	Typ SQL	Ograniczenie	Opis
id	BIGSERIAL	PK	Identyfikator pracownika
user_id	BIGINT	FK → users, UNIQUE	Powiązane konto logowania
company_id	BIGINT	FK → companies	Firma pracodawcy
first_name	VARCHAR(100)	NOT NULL	Imię (WF.02)
last_name	VARCHAR(100)	NOT NULL	Nazwisko (WF.02)
pesel	VARCHAR(11)	UNIQUE	PESEL (WF.02)
address	TEXT		Adres zamieszkania (WF.02)
birth_date	DATE	NOT NULL	Data urodzenia – obliczenia emerytury
retirement_age	INTEGER	DEFAULT 65	Planowany wiek emerytalny (WF.12)
zus_title_code	VARCHAR(6)		Kod tytułu ubezpieczenia ZUS (WF.02)
gross_salary	NUMERIC(12,2)	NOT NULL	Wynagrodzenie zasadnicze brutto
tax_relief	BOOLEAN	DEFAULT TRUE	Ulga podatkowa – złożony PIT-2

4.5 Tabela: absences (WF.25)

Oddzielna tabela nieobecności umożliwia zarządzanie urlopami i L4 niezależnie od generowania paska (WF.25).

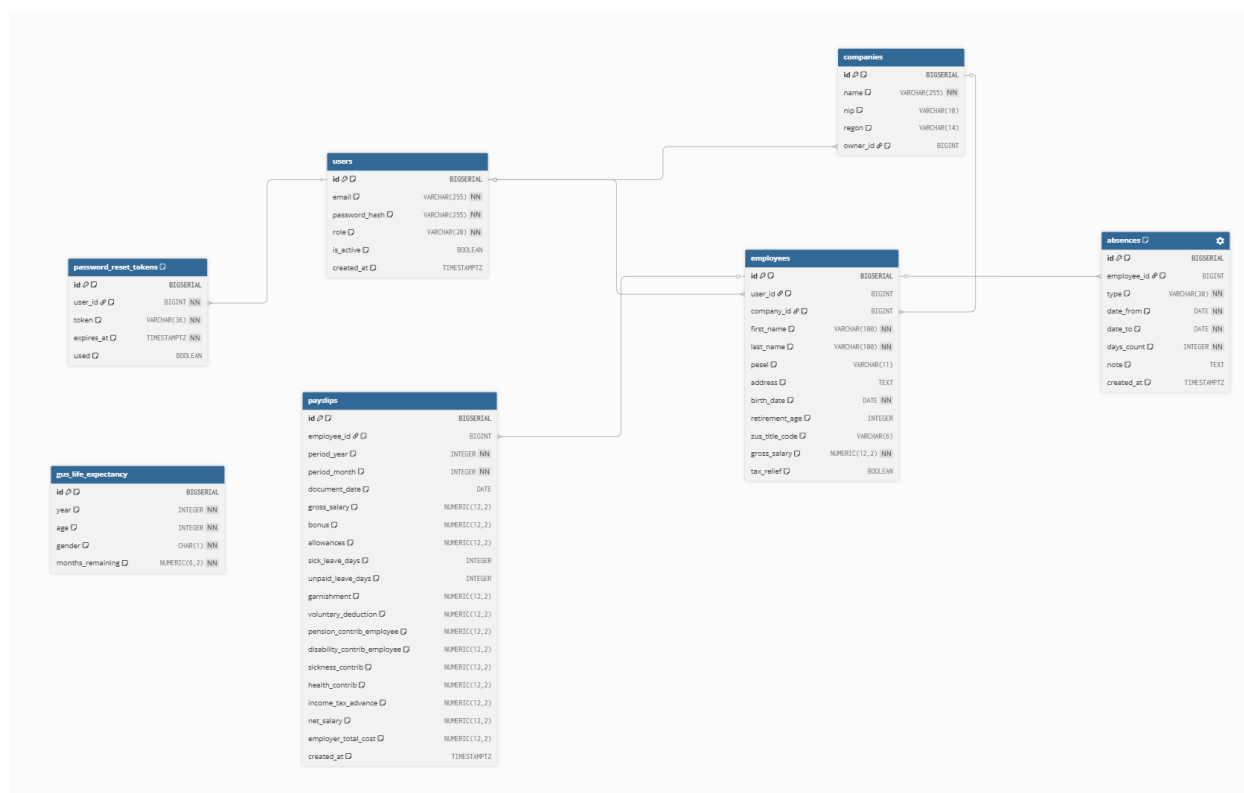
Kolumna	Typ SQL	Ograniczenie	Opis
id	BIGSERIAL	PK	Identyfikator
employee_id	BIGINT	FK → employees	Pracownik
type	VARCHAR(30)	NOT NULL	'SICK_LEAVE', 'UNPAID_LEAVE', 'PAID_LEAVE', 'MATERNITY'
date_from	DATE	NOT NULL	Pierwsza data nieobecności
date_to	DATE	NOT NULL	Ostatnia data nieobecności
days_count	INTEGER	NOT NULL	Liczba dni roboczych – obliczona przy zapisie
note	TEXT		Opcjonalna notatka (np. numer L4)
created_at	TIMESTAMPTZ	DEFAULT NOW()	Data wprowadzenia przez pracodawcę

4.6 Tabela: payslips

Kolumna	Typ SQL	Opis
id	BIGSERIAL PK	Identyfikator paska
employee_id	BIGINT FK → employees	Pracownik (WF.15)
period_year	INTEGER NOT NULL	Rok okresu rozliczeniowego (WF.03)
period_month	INTEGER NOT NULL (1–12)	Miesiąc okresu rozliczeniowego (WF.03)
document_date	DATE	Data sporządzenia dokumentu (WF.03)
gross_salary	NUMERIC(12,2)	Wynagrodzenie zasadnicze
bonus	NUMERIC(12,2) DEFAULT 0	Premia (WF.04)
allowances	NUMERIC(12,2) DEFAULT 0	Dodatki (WF.04)
sick_leave_days	INTEGER DEFAULT 0	Dni L4 – korekta ZUS (WF.05, WF.09)
unpaid_leave_days	INTEGER DEFAULT 0	Dni urlopu bezpłatnego (WF.09)
garnishment	NUMERIC(12,2) DEFAULT 0	Potrącenie komornicze (WF.07)
voluntary_deduction	NUMERIC(12,2) DEFAULT 0	Dobrowolne potrącenie (WF.07)
pension_contrib_employee	NUMERIC(12,2)	Składka emerytalna pracownika – 9,76%
disability_contrib_employee	NUMERIC(12,2)	Składka rentowa pracownika – 1,50%
sickness_contrib	NUMERIC(12,2)	Składka chorobowa – 2,45%
health_contrib	NUMERIC(12,2)	Składka zdrowotna – 9,00%
income_tax_advance	NUMERIC(12,2)	Zaliczka na podatek PIT (WF.06)
net_salary	NUMERIC(12,2)	Wynagrodzenie netto do wypłaty (WF.10)
employer_total_cost	NUMERIC(12,2)	Całkowity koszt pracodawcy (WF.08)
created_at	TIMESTAMPZ DEFAULT NOW()	Archiwizacja – rekord nieusuwalne (WNF.08)

4.7 Tabela: gus_life_expectancy

Kolumna	Typ SQL	Opis
id	BIGSERIAL PK	Identyfikator
year	INTEGER NOT NULL	Rok publikacji tablicy GUS – aktualizacja corocznie (WF.13)
age	INTEGER NOT NULL	Wiek (lata)
gender	CHAR(1) NOT NULL	'M' – mężczyzna, 'F' – kobieta
months_remaining	NUMERIC(6,2) NOT NULL	Przeciętna liczba miesięcy dalszego trwania życia



Rysunek 1. Schemat bazy danych

5. Specyfikacja REST API

Prefix: /api/v1. Autoryzacja: sesja HTTP (cookie JSESSIONID). Format: JSON. Błędy: { "error": "opis", "status": 400 }.

5.1 Autentykacja i reset hasła (WF.17)

Metoda	Endpoint	Rola	Opis
POST	/api/v1/auth/login	Publiczny	Body: { email, password }. Tworzy sesję HTTP. Obsługa błędnych danych z komunikatem.
POST	/api/v1/auth/logout	Zalogowany	Unieważnia sesję. Zwraca HTTP 200.
GET	/api/v1/auth/me	Zalogowany	Dane zalogowanego użytkownika (id, email, rola).
POST	/api/v1/auth/forgot-password	Publiczny	Body: { email }. Generuje token resetujący, wysyła e-mail przez Spring Mail. (WF.17)
POST	/api/v1/auth/reset-password	Publiczny	Body: { token, newPassword }. Weryfikuje token, ustawia nowe hasło. Token staje się używany. (WF.17)

5.2 Pracownicy (WF.22, WF.23)

Metoda	Endpoint	Rola	Opis
GET	/api/v1/employees?search=&sort=	EMPLOYER	Lista pracowników z wyszukiwaniem i sortowaniem (WF.22). Query: search=Kowalski, sort=lastName,asc.
POST	/api/v1/employees	EMPLOYER	Dodanie pracownika + konto users. Body: EmployeeCreateDto.
GET	/api/v1/employees/{id}	EMPLOYER / własny	Profil pracownika. Pracownik widzi tylko swój (WO.03).
PATCH	/api/v1/employees/{id}	EMPLOYER	Edycja danych płacowych: wynagrodzenie, kody ZUS, ulgi, potrącenia (WF.23).
DELETE	/api/v1/employees/{id}	EMPLOYER	Dezaktywacja – is_active=false. Dane pozostają (soft delete).

5.3 Nieobecności (WF.25)

Metoda	Endpoint	Rola	Opis
GET	/api/v1/absences?employeeId=&year=&month=	EMPLOYER / własny	Lista nieobecności dla pracownika w danym okresie. Pracownik widzi tylko swoje.
POST	/api/v1/absences	EMPLOYER	Dodanie nieobecności. Body: { employeeId, type, dateFrom, dateTo, note }. System liczy dni robocze.
DELETE	/api/v1/absences/{id}	EMPLOYER	Usunięcie nieobecności (jeśli pasek za ten okres nie jest jeszcze wygenerowany).

5.4 Paski płacowe (WF.18, WF.19, WF.21, WF.24)

Metoda	Endpoint	Rola	Opis
GET	/api/v1/payslips?employeeId=&year=&month=	EMPLOYER / własny	Historia pasków z filtrowaniem po roku i miesiącu (WF.19). Pracownik widzi tylko swoje.
POST	/api/v1/payslips	EMPLOYER	Generowanie paska. Nieobecności z tabeli absences automatycznie uwzględniane. Podgląd przed zapisem: ?preview=true (WF.24).
GET	/api/v1/payslips/{id}	EMPLOYER / własny	Szczegóły paska – pełny rozkład składników (WF.18).
GET	/api/v1/payslips/{id}/pdf	EMPLOYER / własny	Pobranie paska jako PDF (iText 7). Content-Disposition: attachment (WF.16, WF.21).

5.5 Emerytura (WF.11, WF.12, WF.14, WF.20)

Metoda	Endpoint	Rola	Opis
GET	/api/v1/retirement/{employeeId}/current	EMPLOYER / własny	Bieżąca szacunkowa emerytura (WF.11). Zwraca też tablicę skumulowanych składek miesięcznie do wykresu (WF.20).
POST	/api/v1/retirement/{employeeId}/forecast	EMPLOYER / własny	Prognoza. Body: { targetRetirementAge, ofeAmount }. Dane OFE NIE są zapisywane (WNF.04). Zwraca futurePension (WF.12, WF.14).

6. Logika Obliczeń Płacowych

Wszystkie obliczenia w klasie PayrollService. Wartości: BigDecimal z RoundingMode.HALF_UP – nigdy double ani float (WNF.10).

6.1 Składki ZUS – pracownik (stawki 2025)

Składka	Stawka	Uwagi
Emerytalna (pracownik)	9,76%	Limit roczny ok. 260 190 zł
Rentowa (pracownik)	1,50%	Limit roczny ok. 260 190 zł
Chorobowa	2,45%	Brak limitu. Przy L4: = 0 za dni zwolnienia (WF.05)
Zdrowotna	9,00%	Podstawa: brutto – składki społeczne pracownika

6.2 Składki ZUS – pracodawca (do kosztu WF.08)

Składka	Stawka	Uwagi
Emerytalna (pracodawca)	9,76%	Wchodzi do employer_total_cost
Rentowa (pracodawca)	6,50%	Wchodzi do employer_total_cost
Wypadkowa	1,67%	Stawka uproszczona standardowa
Fundusz Pracy	2,45%	Wchodzi do employer_total_cost
FGŚP	0,10%	Fundusz Gwarantowanych Świadczeń Pracowniczych

6.3 Algorytm obliczenia wynagrodzenia netto

```
KROK 1: Brutto bazowe (uwzględnienie nieobecności WF.09)
bruttoBase = grossSalary + bonus + allowances
jeśli unpaidLeaveDays > 0:
    bruttoBase -= (grossSalary / roboczeDniWMiesiacu) * unpaidLeaveDays
jeśli sickLeaveDays > 0:
    // wynagrodzenie chorobowe = 80% za dni L4, podstawa ZUS skorygowana (WF.05)
    sickPay = (grossSalary / 30) * 0.80 * sickLeaveDays
    bruttoBase = bruttoBase - (grossSalary / 30 * sickLeaveDays) + sickPay

KROK 2: Składki społeczne pracownika
pensionEmployee = bruttoBase * 9,76%
disabilityEmployee = bruttoBase * 1,50%
sicknessEmployee = bruttoBase * 2,45% (0 za dni L4)
socialTotal = pensionEmployee + disabilityEmployee + sicknessEmployee

KROK 3: Składka zdrowotna
healthBase = bruttoBase - socialTotal
healthContrib = healthBase * 9,00%

KROK 4: Zaliczka PIT (WF.06)
taxBase = ROUND(bruttoBase - socialTotal - 250, 0) [koszty uzyskania przychodu]
stawka = taxBase <= próg roczny ? 12% : 32%
taxBeforeRelief = taxBase * stawka
```

```
jeśli taxRelief = true: taxBeforeRelief -= 300 PLN [ulga miesięczna PIT-2]
healthDeductible = healthBase × 7,75%
incomeTaxAdvance = MAX(ROUND(taxBeforeRelief - healthDeductible, 0), 0)
```

KROK 5: Netto (WF.10)

```
netSalary = bruttoBase - socialTotal - healthContrib
            - incomeTaxAdvance - garnishment - voluntaryDeduction
```

KROK 6: Koszt pracodawcy (WF.08)

```
employerTotalCost = bruttoBase
+ (bruttoBase × 9,76%) // emerytalna pracodawca
+ (bruttoBase × 6,50%) // rentowa pracodawca
+ (bruttoBase × 1,67%) // wypadkowa
+ (bruttoBase × 2,45%) // Fundusz Pracy
+ (bruttoBase × 0,10%) // FGŚP
```

6.4 Obliczenie emerytury i wykresu (WF.11–14, WF.20)

Bieżąca emerytura (WF.11):

```
sumContribs = SUM(pension_contrib_employee) FROM payslips WHERE employee_id = X
gusMonths    = months_remaining FROM gus_life_expectancy
              WHERE age = currentAge AND year = latestYear
currentPension = sumContribs / gusMonths
```

Dane do wykresu skumulowanych składek (WF.20):

Zapytanie JPQL grupujące sumy składek miesięcznie:

```
SELECT period_year, period_month, SUM(pension_contrib_employee)
FROM payslips WHERE employee_id = X
GROUP BY period_year, period_month ORDER BY period_year, period_month
→ endpoint zwraca tablicę [{year, month, cumulative}] dla Chart.js
```

Prognoza emerytury (WF.12, WF.14):

```
monthsToRetirement = (retirementAge - currentAge) × 12
futureContribs       = grossSalary × 9,76% × monthsToRetirement
totalContribs        = sumContribs + futureContribs + ofeAmount (opcjonalnie)
gusAtRetirement     = months_remaining WHERE age = retirementAge
futurePension        = totalContribs / gusAtRetirement
```

7. Frontend – HTML / CSS / JavaScript

Pliki statyczne serwowane przez Spring Boot z katalogu src/main/resources/static/. Bootstrap 5 z CDN. Komunikacja z API przez Fetch API z credentials: 'include'.

7.1 Ekrany aplikacji

Ekran / Plik	Funkcjonalność	Sygnatura
login.html	Formularz logowania + obsługa błędów + link do resetu hasła	WF.17
forgot-password.html	Formularz podania e-mail do resetu hasła	WF.17
reset-password.html	Formularz nowego hasła (token z URL)	WF.17
employee-dashboard.html	Ostatni pasek, skrót netto/brutto, nawigacja	WF.18
payslip-detail.html	Pełny rozkład paska: brutto, ZUS, PIT, netto	WF.18
payslips.html	Historia z filtrem rok/miesiąc, pobieranie PDF	WF.19, WF.21
retirement.html	Bieżąca emerytura + prognoza + pole OFE + wykres Chart.js	WF.20, WF.14
employer-dashboard.html	Lista pracowników, skróty do generowania pasków	WF.22
employees.html	Tabela z wyszukiwarką i sortowaniem (kolumna + kierunek)	WF.22
employee-form.html	Formularz edycji: wynagrodzenie, ZUS, ulgi, potrącenia	WF.23
absences.html	Tabela nieobecności, formularz dodawania urlop/L4	WF.25
payslip-form.html	Wybór pracownika i okresu, podgląd wyniku przed zapisem	WF.24

7.2 Wykres skumulowanych składek (WF.20 – Chart.js)

```
// retirement.js - wykres skumulowanych składek emerytalnych
const response = await fetch(
  `/api/v1/retirement/${employeeId}/current`,
  { credentials: 'include' }
);
const data = await response.json();

// data.monthlyContributions = [{year, month, cumulative}, ...]
const labels = data.monthlyContributions.map(
  c => `${c.year}-${String(c.month).padStart(2, '0')}`
);
const values = data.monthlyContributions.map(c => c.cumulative);

new Chart(document.getElementById('retirementChart'), {
  type: 'line',
  data: {
    labels,
    datasets: [{
      label: 'Skumulowane składki emerytalne (ZUS I filar)',
      data: values,
      borderColor: '#1A5276',
      fill: true
    }]
  }
});
```

8. Proponowana Struktura Projektu

Poniższa struktura katalogów stanowi propozycję organizacji kodu. Zespół implementacyjny może ją dostosować do własnych potrzeb, zachowując jednak podział na warstwy kontroler / serwis / repozytorium / encja / dto.

```
kalkulator-placowy/
├── pom.xml
├── src/main/
│   ├── java/pl/edu/payroll/
│   │   ├── PayrollApplication.java
│   │   ├── config/
│   │   │   └── SecurityConfig.java
│   │   ├── controller/
│   │   │   ├── AuthController.java           // login, logout, forgot/reset password
│   │   │   ├── EmployeeController.java      // CRUD + search + sort (WF.22,23)
│   │   │   ├── AbsenceController.java       // (WF.25)
│   │   │   ├── PayslipController.java       // historia, generowanie, PDF
│   │   │   └── RetirementController.java    // bieżąca, prognoza, wykres
│   │   ├── service/
│   │   │   ├── PayrollService.java          // obliczenia ZUS + PIT
│   │   │   ├── RetirementService.java       // emerytura + dane do wykresu
│   │   │   ├── AbsenceService.java
│   │   │   ├── PasswordResetService.java    // (WF.17)
│   │   │   ├── EmployeeService.java
│   │   │   └── PdfService.java              // iText 7
│   │   ├── repository/
│   │   │   ├── UserRepository.java
│   │   │   ├── EmployeeRepository.java
│   │   │   ├── AbsenceRepository.java
│   │   │   ├── PasswordResetTokenRepository.java
│   │   │   ├── PayslipRepository.java
│   │   │   └── GusRepository.java
│   │   ├── entity/
│   │   │   ├── User.java
│   │   │   ├── Company.java
│   │   │   ├── Employee.java
│   │   │   ├── Absence.java
│   │   │   ├── PasswordResetToken.java
│   │   │   ├── Payslip.java
│   │   │   └── GusLifeExpectancy.java
│   │   ├── dto/
│   │   │   ├── PayslipRequestDto.java
│   │   │   ├── PayslipResponseDto.java
│   │   │   ├── AbsenceDto.java
│   │   │   ├── RetirementResponseDto.java   // zawiera monthlyContributions[]
│   │   │   └── EmployeeCreateDto.java
│   └── resources/
│       ├── application.properties
│       ├── db/migration/
│       │   ├── V1__init_schema.sql
│       │   ├── V2__add_absences.sql
│       │   ├── V3__add_password_reset.sql
│       │   └── V4__seed_gus_2025.sql
│       ├── static/
│       │   ├── login.html
│       │   ├── forgot-password.html
│       │   ├── reset-password.html
│       │   ├── absences.html
│       │   ├── payslips.html
│       │   ├── retirement.html
│       │   ├── employees.html
│       │   ├── css/style.css
│       └── js/
```



```
|— auth.js
|— employees.js
|— absences.js
|— payslips.js
|— retirement.js
```

9. Środowisko i Uruchomienie

Narzędzie	Wymagana wersja
JDK	Java 21 LTS (np. Eclipse Temurin)
Maven	3.9.x (lub dołączony ./mvnw)
PostgreSQL	16.x – lokalnie lub Docker
SMTP (reset hasła)	Konto Gmail / Mailtrap.io (dev) – konfiguracja w application.properties
IDE	IntelliJ IDEA Community lub Eclipse + Spring Tools
Przeglądarka	Chrome, Firefox, Edge, Safari (WNF.07)

9.1 application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/payroll_db
spring.datasource.username=postgres
spring.datasource.password=tajnehaslo
spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
server.port=8080
server.servlet.session.timeout=30m

# Spring Mail - reset hasła (WF.17)
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=twoj@gmail.com
spring.mail.password=app-password
spring.mail.properties.mail.smtp.starttls.enable=true

app.base-url=http://localhost:8080
app.password-reset.token-expiry-hours=1
```

9.2 Uruchomienie

```
# 1. Utwórz bazę danych
createdb payroll_db

# 2. Uruchom - Flyway wykona migracje automatycznie
./mvnw spring-boot:run

# 3. Otwórz w przeglądarce
http://localhost:8080/login.html

# 4. Testy
./mvnw test
```

10. Termin Realizacji

Implementację systemu Kalkulator Płacowy należy wykonać i oddać do 31 maja 2026 roku. Termin obejmuje działającą aplikację spełniającą wszystkie wymagania funkcjonalne i pozafunkcjonalne określone w Części I niniejszego dokumentu.

11. Lista Kontrolna przed Oddaniem

Pokrycie wymagań

Sygnatura	Co sprawdzić	Gdzie zaimplementowane
WO.01	Aplikacja działa jako samodzielna aplikacja webowa z bazą danych	Spring Boot + PostgreSQL, plik pom.xml, application.properties
WO.02	Istnieją dwa rodzaje kont: pracodawca i pracownik	Encja User, pole role ('EMPLOYER'/'EMPLOYEE'), SecurityConfig
WO.03	Pracownik widzi tylko swoje dane i nie może ich edytować; pracodawca widzi wszystkich i może edytować	Weryfikacja userId w serwisach, @PreAuthorize w kontrolerach
WO.04	System obsługuje wyłącznie walutę PLN – brak pól walutowych, brak przeliczników	Encja Payslip, wszystkie pola NUMERIC bez waluty
WO.05	Dane pracownika do kalkulacji wprowadza i aktualizuje pracodawca (nie pracownik)	PATCH /employees/{id} dostępny tylko dla EMPLOYER
WF.01	Dane firmy (nazwa, NIP, REGON) widoczne na pasku płacowym pracownika	Encja Company, PayslipResponseDto zawiera dane firmy, payslip-detail.html
WF.02	System przechowuje dane osobowe pracownika: imię, nazwisko, PESEL, adres, kody ZUS	Encja Employee, tabela employees, employee-form.html
WF.03	Można określić datę sporządzenia dokumentu i okres rozliczeniowy (rok + miesiąc)	Pola document_date, period_year, period_month w tabeli payslips, payslip-form.html
WF.04	Obliczanie składników brutto (zasadnicze, premia, dodatki) i składek ZUS pracownika i pracodawcy	PayrollService – kroki 1–2 i 6 algorytmu
WF.05	Zasiłki (L4, macierzyński) korygują podstawę składek ZUS	PayrollService – obsługa sick_leave_days, korekta socialTotal
WF.06	Zaliczka PIT naliczana z progami (12%/32%), ulgą kwoty wolnej i kosztami uzyskania	PayrollService – krok 4 algorytmu
WF.07	Potrącenia komornicze i dobrowolne uwzględnione w kwocie netto	Pola garnishment, voluntary_deduction w Payslip, krok 5 algorytmu
WF.08	Całkowity koszt pracodawcy = brutto + składki pracodawcy	PayrollService – krok 6, pole employer_total_cost w Payslip

WF.09	Urlopy płatne i bezpłatne korygują wynagrodzenie za dany okres	PayrollService – krok 1, tabela absences, AbsenceService
WF.10	Wynagrodzenie netto (do wypłaty) wyświetlane pracownikowi	Pole net_salary w Payslip, payslip-detail.html
WF.11	Szacowana bieżąca emerytura na podstawie sumy dotychczasowych składek emerytalnych	RetirementService, GET /retirement/{id}/current
WF.12	Szacowana przyszła emerytura z uwzględnieniem wieku i planowanej daty przejścia na emeryturę	RetirementService, POST /retirement/{id}/forecast
WF.13	Wskaźnik GUS dalszego trwania życia stosowany w obliczeniach, aktualizowany corocznie	Tabela gus_life_expectancy, migracja V4__seed_gus_2025.sql, GusRepository
WF.14	Rozróżnienie składek ZUS (I filar, auto) i OFE (II filar, ręcznie) w kalkulatorze emerytalnym	RetirementForecastDto – pole ofeAmount, RetirementService – obliczenie totalContribs
WF.15	Historia pasków płacowych przechowywana i dostępna dla pracownika	Tabela payslips, GET /payslips?employeeId=, payslips.html
WF.16	Eksport paska do PDF możliwy do pobrania	PdfService (iText 7), GET /payslips/{id}/pdf, Content-Disposition: attachment
WF.17	Panel logowania z obsługą błędów i resetem hasła przez e-mail	AuthController, PasswordResetService, login.html, forgot-password.html, reset-password.html
WF.18	Widok bieżącego paska: składniki brutto, potrącenia, netto	payslip-detail.html, GET /payslips/{id}
WF.19	Historia wypłat z filtrem po roku i miesiącu	payslips.html, GET /payslips?year=&month=
WF.20	Kalkulator emerytalny z wykresem skumulowanych składek	retirement.html, Chart.js, RetirementResponseDto.monthlyContributions
WF.21	Pobieranie wybranego paska jako PDF	GET /payslips/{id}/pdf, przycisk pobierz w payslips.html
WF.22	Lista pracowników z wyszukiwaniem i sortowaniem	employees.html, GET /employees?search=&sort=
WF.23	Karta pracownika – edycja danych płacowych	employee-form.html, PATCH /employees/{id}
WF.24	Generowanie paska: wybór okresu, inicjowanie obliczeń, podgląd wyniku	payslip-form.html, POST /payslips?preview=true
WF.25	Zarządzanie nieobecnościami: urlopy, L4, inne	absences.html, AbsenceController, tabela absences
WNF.01	Czas generowania paska < 10 sekund od żądania	Test wydajnościowy: zmierzyć czas POST /payslips dla typowego pracownika
WNF.02	Komunikacja wyłącznie przez HTTPS (TLS 1.2+)	Konfiguracja server.ssl.* lub Nginx jako reverse proxy

WNF.03	Hasła w bazie wyłącznie jako hash BCrypt, strength ≥ 12	BCryptPasswordEncoder w SecurityConfig, weryfikacja w bazie: brak jawnych haseł
WNF.04	Dane OFE nie zapisywane do bazy – tylko obliczenia in-memory	RetirementController: ofeAmount tylko w DTO, brak mapowania do encji ani do bazy
WNF.05	Dostęp do danych kontrolowany przez RBAC; pracownik nie widzi danych innych pracowników	@PreAuthorize, weryfikacja userId w serwisach, testy integracyjne ról
WNF.06	Interfejs intuicyjny, obsługa bez szkolenia	Przegląd UI: etykiety pól, komunikaty błędów, nawigacja
WNF.07	Aplikacja działa w Chrome, Firefox, Edge i Safari	Testy manualne w każdej przeglądarce
WNF.08	Paski płacowe archiwizowane, nieusuwalne przez co najmniej 5 lat	Brak metody delete w PayslipRepository, brak endpointu DELETE /payslips
WNF.09	System zgodny z RODO – brak zbędnych logów z danymi osobowymi	Sprawdzić logi aplikacji: brak PESEL/wynagrodzenia w logach
WNF.10	Obliczenia zgodne z aktualnymi przepisami PIT i ZUS	Weryfikacja stawek w PayrollService z obowiązującymi przepisami przed oddaniem

Bezpieczeństwo

- Hasła hashowane BCrypt strength ≥ 12 , żadne hasło jawne w bazie (WNF.03)
- RBAC: pracownik nie ma dostępu do danych innych pracowników (WNF.05)
- Tokeny resetu hasła: UUID, jednorazowe, ważne 1h (WF.17)
- Dane OFE nie zapisywane do bazy – obliczenia in-memory (WNF.04)
- Paski płacowe nieusuwalne – brak fizycznego DELETE (WNF.08)
- HTTPS/TLS lub opis planu wdrożenia (WNF.02)

Jakość kodu

- Wszystkie kwoty pieniężne: BigDecimal z RoundingMode.HALF_UP – nigdy double/float
- Stawki ZUS i progi PIT zweryfikowane przed oddaniem (WNF.10)
- Wskaźnik GUS w bazie – tabela gus_life_expectancy z migracji V4 (WF.13)
- Testy jednostkowe logiki płacowej: próg 12%, 32%, L4, potrącenia, OFE
- Aplikacja działa w Chrome, Firefox, Edge, Safari (WNF.07)